

LOCAL LAB MONITORING USING SPLUNK

SUBMITTED BY:

MUBEENA V

SUBMITTED TO:

GLADWIN

INDEX

Sl.No	Content	Page.No
1	Introduction	3
2	Objectives	3
3	Tools and Technologies	3
4	Lab Setup	4-5
5	Configure Splunk Forwarder in Receiver	5-6
6	Configure Data Inputs in Splunk	7
7	Configure TCP/UDP Inputs in Splunk	8
8	Configure Input Settings	9-10
9	Create Brute-Force Detection Alert	11-12
10	Create Batch Script for IP Blocking	12
11	Enable and Verify SSH Service	13
12	Simulate Brute-Force Attack Using Hydra	14
13	Triggered Alert Detection in Splunk	15
14	Conclusion	16
15	Future Enhancements	16

INTRODUCTION

The objective of this project is to demonstrate how **Splunk Enterprise** can be used to detect brute-force login attempts in a virtual lab environment. In this setup, failed login attempts are collected from a Windows system, monitored in Splunk, and analyzed to identify suspicious activity.

When repeated failed login attempts are detected, Splunk generates an alert and triggers a response to block the attacker's IP address using Windows Firewall. This project shows how Splunk can be used for log monitoring, attack detection, and automated response in a simple security monitoring system.

OBJECTIVES

Configure a Windows 10 machine with Splunk Universal Forwarder to collect security logs.

Forward logs to a Windows 11 system running the Splunk server.

Simulate an SSH / RDP brute-force attack against Windows 10.

Create correlation rules in Splunk to detect suspicious login behavior.

Generate alerts for security incidents.

Implement basic prevention mechanisms based on detected attacks.

TOOLS AND TECHNOLOGIES

Tool	Purpose
Splunk Enterprise (Win 11)	Log monitoring and alerting
Splunk Universal Forwarder	Log forwarding from client (Win 10)
Kali Linux	Launching brute-force attacks
Hydra	Simulating brute-force login attempts
Windows Firewall	Manual IP blocking
VirtualBox	Virtual lab environment

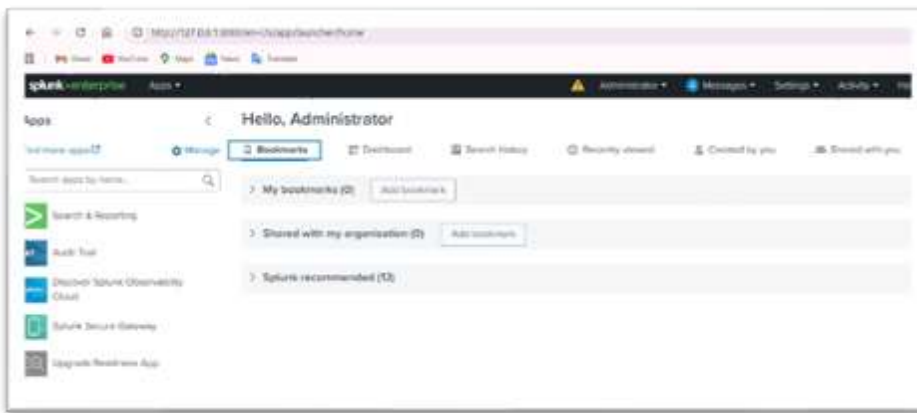
LAB SETUP

The lab environment was designed using three separate systems to simulate a real-world brute-force detection scenario. Each machine had a specific role in the attack, monitoring, and response workflow.

Host System (Windows 11)

The host machine was used as the central monitoring system.

It had **Splunk Enterprise** installed and acted as the **SIEM server** responsible for collecting, indexing, analyzing, and visualizing logs received from the client machine.

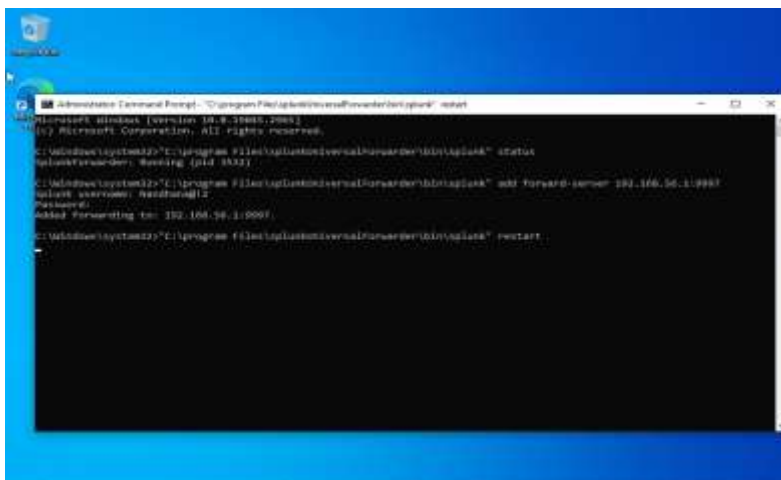


This system was also used to configure detection rules, alerts, and automated response actions.

Client System (Windows 10 Virtual Machine)

The client machine acted as the target system for the brute-force attack.

It had **Splunk Universal Forwarder** installed and configured to forward **Windows Security logs** to the Splunk Enterprise server running on the host system.



The forwarded logs included important authentication events such as:

- * Successful login attempts
- * Failed login attempts
- * User account activity

This machine was intentionally exposed to simulated login attempts so that attack activity could be captured and monitored.

Attacker System (Kali Linux Virtual Machine)

The attacker machine was configured using **Kali Linux**.

This system was used to simulate brute-force attacks against the Windows client machine using tools such as **Hydra**.

The purpose of this machine was to generate repeated failed login attempts, allowing Splunk to detect suspicious authentication behavior and trigger alerts.

Network Configuration

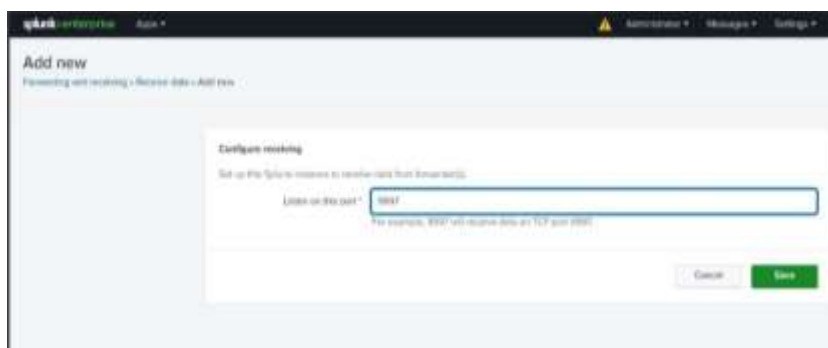
All three systems were connected within the same virtual network environment to allow proper communication between them.

The network was configured to ensure that:

- * The **Kali attacker machine** could reach the **Windows client system**
- * The **Windows client system** could forward logs to the **Splunk Enterprise host**
- * The **Host machine** could receive, analyze, and respond to suspicious events

This setup created a complete end-to-end lab environment for simulating, detecting, and responding to brute-force attacks in real time.

Configure Splunk Forwarder in Receiver

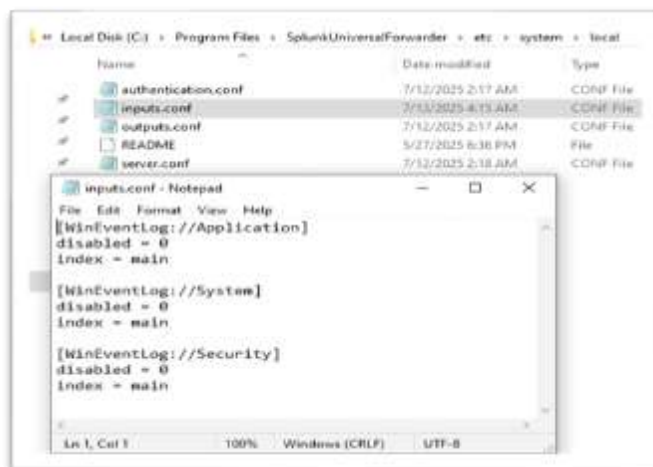


The Windows 10 client system was connected to the Splunk Enterprise server (Windows 11) using **Splunk Universal Forwarder** to send security logs for monitoring.

The following steps were performed:

- * Opened **Splunk Enterprise** on the host system
- * Navigated to **Settings > Forwarding and Receiving**
- * Selected **Configure Receiving** under the receiving section
- * Clicked **Add New** to create a new receiver
- * Configured Splunk to listen on port **9997**
- * Saved the configuration to allow incoming data from the forwarder

Port **9997** was enabled on the Splunk Enterprise server to receive forwarded logs from the Windows 10 client machine. This established communication between the forwarder and the Splunk server for centralized log collection.

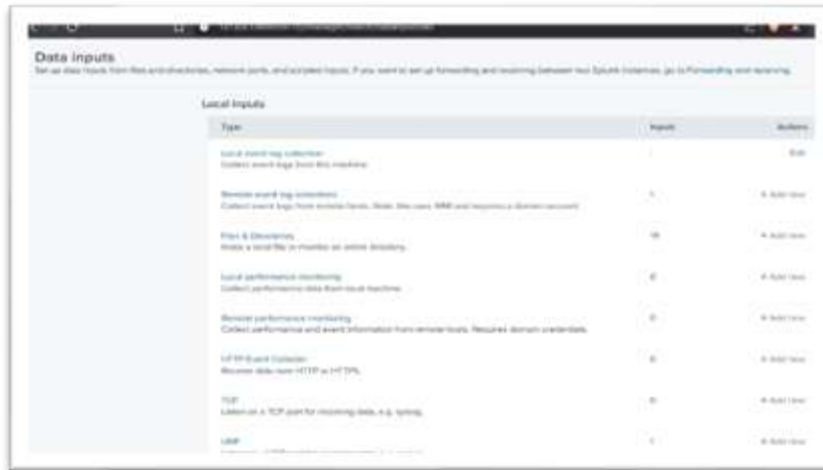


The Splunk Universal Forwarder was configured to collect Windows event logs by editing the inputs.conf file in:

C:\Program Files\SplunkUniversalForwarder\etc\system\local\

The Application, System, and Security event logs were enabled and assigned to the main index. This allowed the client system to forward important Windows logs to the Splunk Enterprise server for monitoring.

Configure Data Inputs in Splunk



To collect and monitor security-related events, data input settings were configured in Splunk Enterprise. This step allows Splunk to receive and process different types of logs from local and remote systems.

The following steps were performed:

- * Opened **Splunk Enterprise** on the host system
- * Navigated to **Settings > Data Inputs**
- * Accessed the **Data Inputs** configuration page
- * Viewed available input sources such as:
 - * Local event log collection
 - * Remote event log collections
 - * Files and directories
 - * TCP and UDP network inputs
- * Verified that required input sources were available for collecting system and security logs

The **Data Inputs** section was used to manage log collection sources in Splunk. This configuration ensured that Splunk was ready to receive and process event data from connected systems for monitoring and analysis.

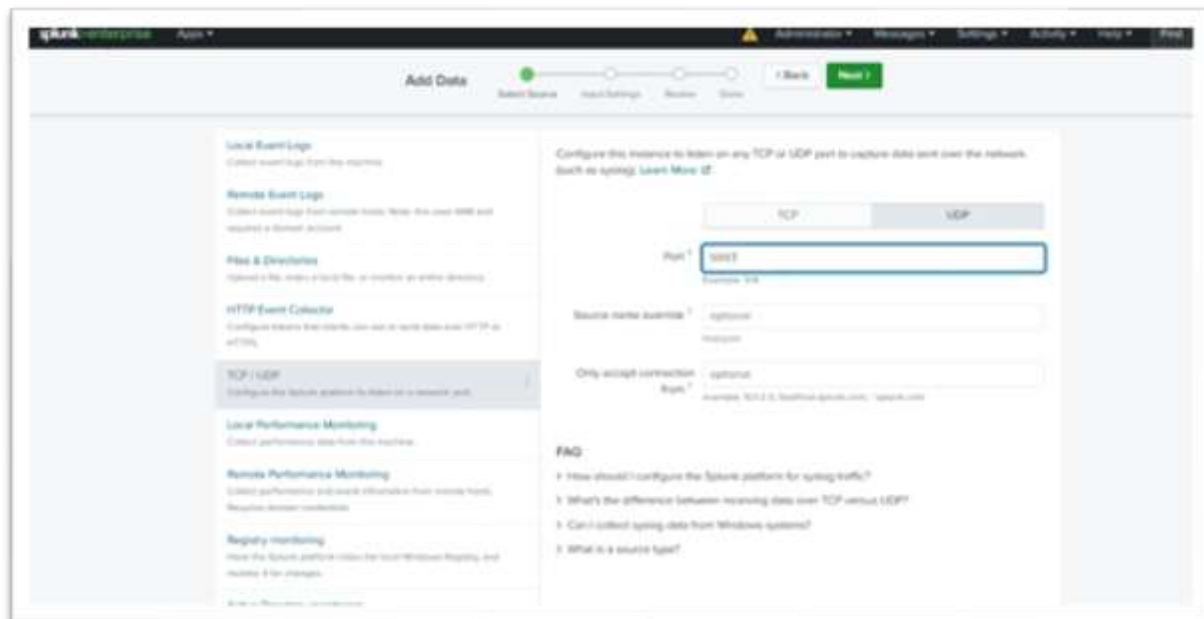
Configure TCP/UDP Input in Splunk

To receive forwarded log data over the network, a TCP/UDP input was configured in Splunk Enterprise. This allows Splunk to listen on a specific network port and accept incoming log data from connected systems.

The following steps were performed:

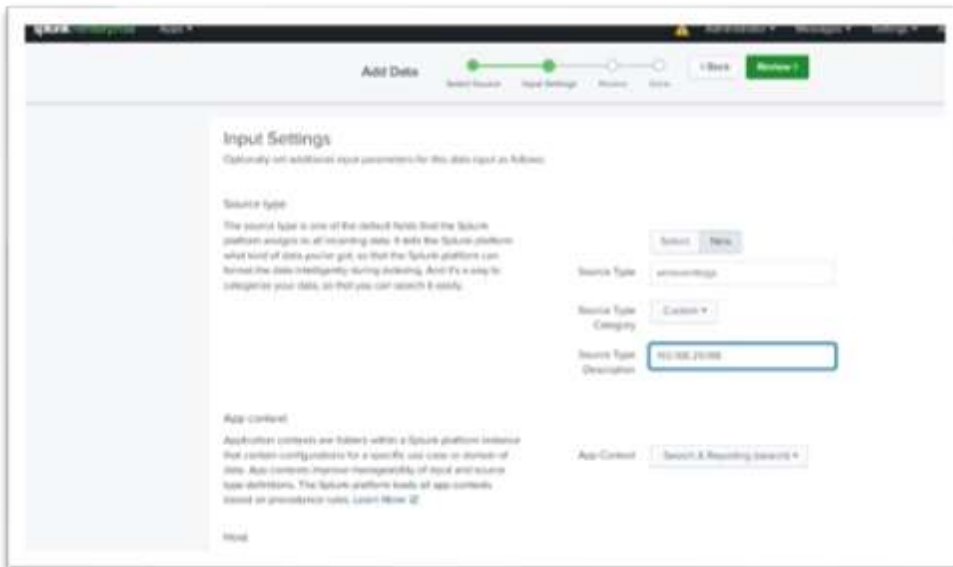
- * Opened **Splunk Enterprise** on the host system
- * Navigated to **Settings > Data Inputs**
- * Selected **TCP / UDP** from the available input options
- * Chose the **UDP** input type
- * Entered port number **9997**
- * Left optional settings as default
- * Proceeded to the next step to complete input configuration

Port **9997** was configured to allow Splunk to listen for incoming forwarded data from connected systems. This enabled the Splunk server to receive log traffic over the network for centralized monitoring and analysis.



Configure Input Settings

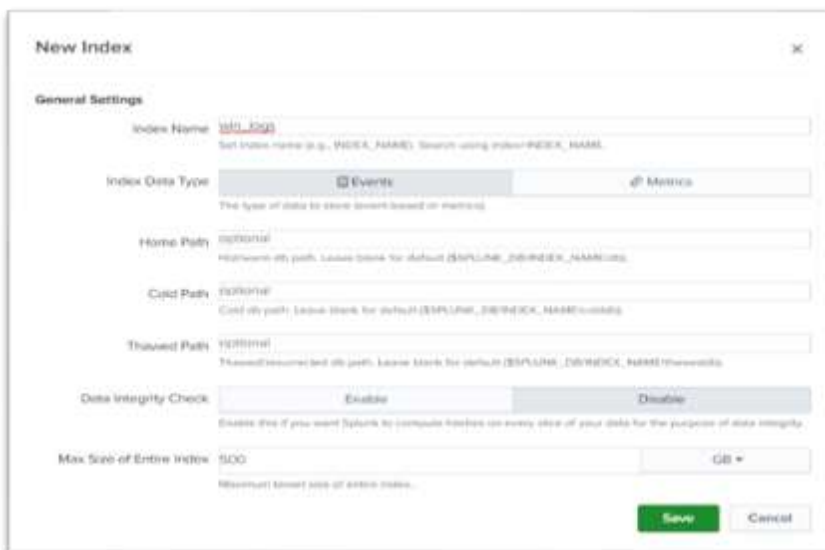
In the Input Settings step, the source type was set to **wineventlog** so Splunk could correctly recognize and process Windows event logs. The source type category was kept as *Custom, and the client system IP address was added in the description field for easier identification of the log source. The app context was set to **Search & Reporting** to manage and store the input configuration within the default Splunk search application.



The screenshot shows the 'Input Settings' page in the Splunk web interface. At the top, there's a progress bar with steps: 'Select Source', 'Input Settings', 'Review', and 'Save'. The 'Input Settings' step is currently active. The page title is 'Input Settings' with a subtitle 'Optionally set additional input parameters for this data input as follows'. Under the 'Source type' section, there's a text block explaining that the source type is one of the default fields that the Splunk platform assigns to all incoming data. To the right, there are input fields: 'Source Type' is set to 'wineventlog', 'Source Type Category' is set to 'Custom', and 'Source Type Description' is set to '10.10.10.10'. Below this, the 'App context' section explains that application contexts are folders within a Splunk platform instance. The 'App Context' field is set to 'Search & Reporting (search)'. At the bottom, there's a 'Next' button.

Create New Index

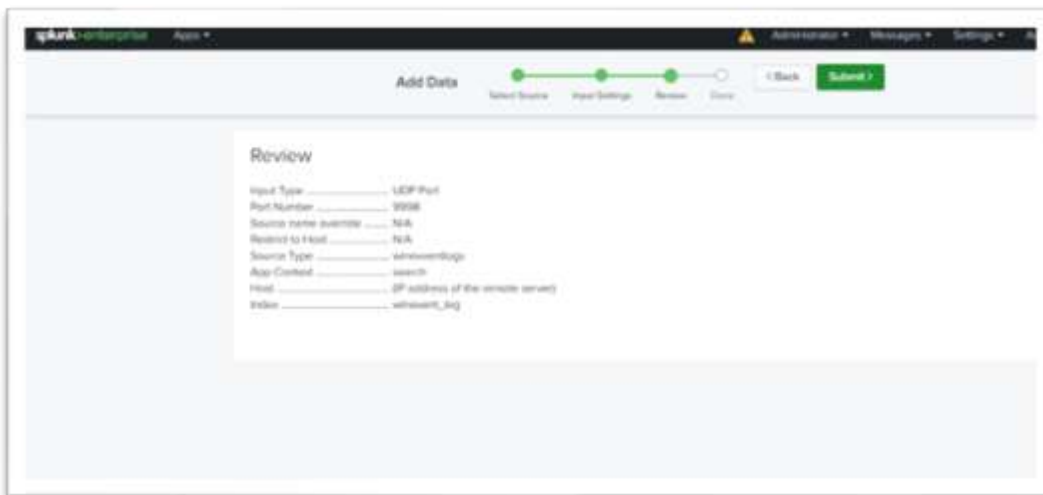
A new index named **win_logs** was created in Splunk to store the forwarded Windows event logs separately. The index type was set to **Events**, while the default storage paths were kept unchanged. This helped organize Windows log data in a dedicated index for easier searching and analysis.



The screenshot shows the 'New Index' dialog box in the Splunk web interface. The 'General Settings' section is visible. The 'Index Name' field is set to 'win_logs'. The 'Index Data Type' is set to 'Events'. The 'Home Path' is set to '\$SPLUNK_HOME/var/lib/splunk/indexes/win_logs'. The 'Cold Path' is set to '\$SPLUNK_HOME/var/lib/splunk/indexes/win_logs/cold'. The 'Thawed Path' is set to '\$SPLUNK_HOME/var/lib/splunk/indexes/win_logs/thawed'. The 'Data Integrity Check' is set to 'Enable'. The 'Max Size of Entire Index' is set to '100 GB'. At the bottom, there are 'Save' and 'Cancel' buttons.

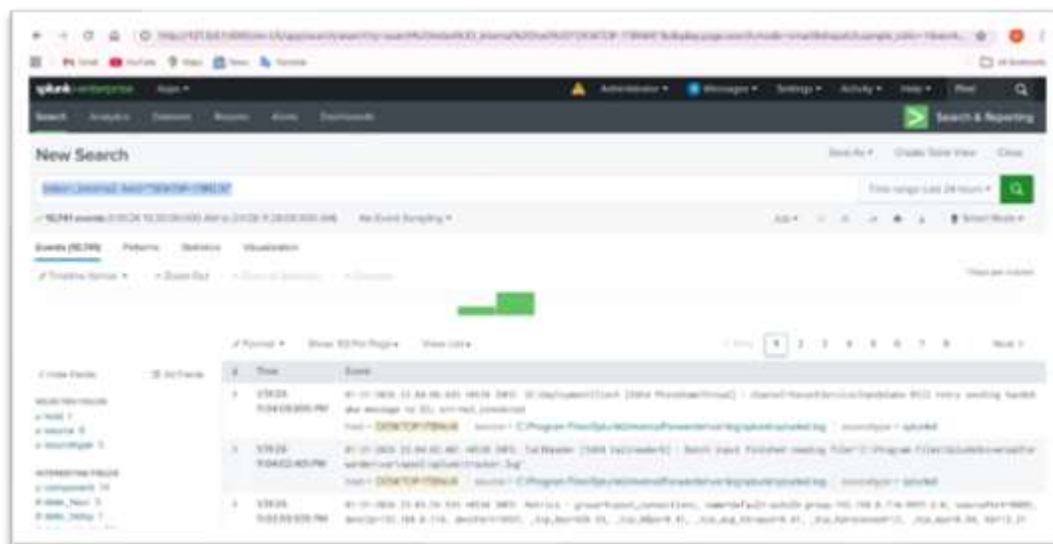
Review and Submit Configuration

In the review step, all input settings were verified before final submission. The configuration included UDP port **9998**, source type **wineventlogs**, app context **search**, and the custom index created for storing Windows event logs. After confirming the settings, the configuration was submitted to complete the data input setup.



Verify Forwarder Connection

The connection between the Splunk Universal Forwarder and Splunk Enterprise was verified by searching internal Splunk logs using the query `index=_internal host="DESKTOP-1TBNIJ6"`. This search displayed log events generated by the Universal Forwarder, including connection activity, log forwarding status, and communication with the Splunk server. The presence of these events confirmed that the forwarder was successfully connected and actively sending data to the Splunk Enterprise server.



Create Brute-Force Detection Alert

To detect repeated failed login attempts automatically, a Splunk alert was created using the failed login search query. The query used was `index=windows EventCode=4625 | stats count by Account_Name, src_ip | where count >= 5`, which checks for multiple failed login attempts from the same source IP address. This helps identify possible brute-force activity targeting Windows accounts.

The search was first tested in Splunk Search & Reporting. After confirming the query, it was saved by selecting **Save As** → **Alert** from the top menu. A new alert was then created with the title “**windows bruteforce attack detected**” and a short description explaining that it detects multiple failed login attempts from the same source IP.

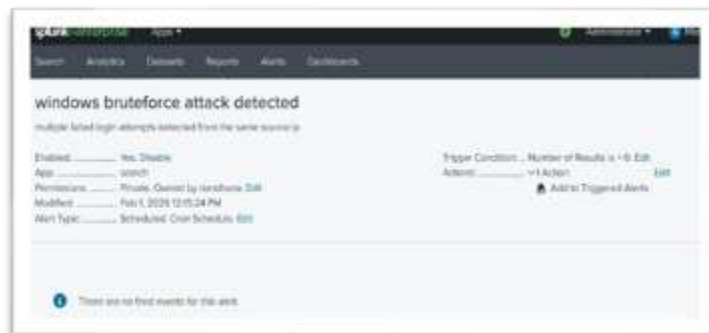
The alert was configured as a **Scheduled Alert**, set to run automatically every 5 minutes using the cron schedule `* / 5 * * * *`. The time range was set to monitor the **last 5 minutes**, allowing Splunk to continuously check for recent failed login activity. This ensured that repeated failed login attempts were detected quickly.

In the trigger settings, the condition was set to activate when the *number of results is greater than 0, meaning the alert would trigger whenever the search found at least one suspicious brute-force pattern. The trigger option was set to **Once**, so the alert would generate a single notification for each detection cycle.

Under Trigger Actions, **Add to Triggered Alerts** was enabled and the severity level was set to **High**. This allowed detected brute-force attempts to appear in Splunk’s alert dashboard as high-priority security events for immediate review.

After saving, Splunk confirmed that the alert was successfully created. The final alert page displayed all configured settings, including alert name, description, schedule, trigger condition, and severity level. This completed the automated brute-force detection setup in Splunk.





Create Batch Script for IP Blocking

To automatically block suspicious IP addresses, a batch script named **block_bruteforce_ip.bat** was created and linked to the Splunk alert. This script is executed whenever the brute-force detection alert is triggered.

The batch file contains a Windows Firewall command that dynamically blocks the attacker's IP address. It takes the detected source IP as input and creates a new inbound firewall rule to deny further connections from that IP.

The script used is shown below:

```
@echo off
set ip=%1
netsh advfirewall firewall add rule name="Splunk Block %ip%" dir=in action=block
remoteip=%ip%
```

Script Explanation

- * @echo off hides command output for cleaner execution.
- * set ip=%1 stores the detected attacker IP passed by Splunk.
- * netsh advfirewall firewall add rule creates a new Windows Firewall rule.
- * action=block blocks traffic from the attacker.
- * remoteip=%ip% applies the rule to the detected source IP.

This script enables automatic



Enable and Verify SSH Service

This screenshot shows the Windows client being prepared for SSH-based brute-force testing. Since Hydra performs attacks over SSH, the **OpenSSH Server** had to be enabled on the Windows client machine.

Using **Windows PowerShell (Administrator)**, the SSH service was started with:

```
powershell id="w7u6u2"
```

```
Start-Service sshd
```

The service status was then checked using:

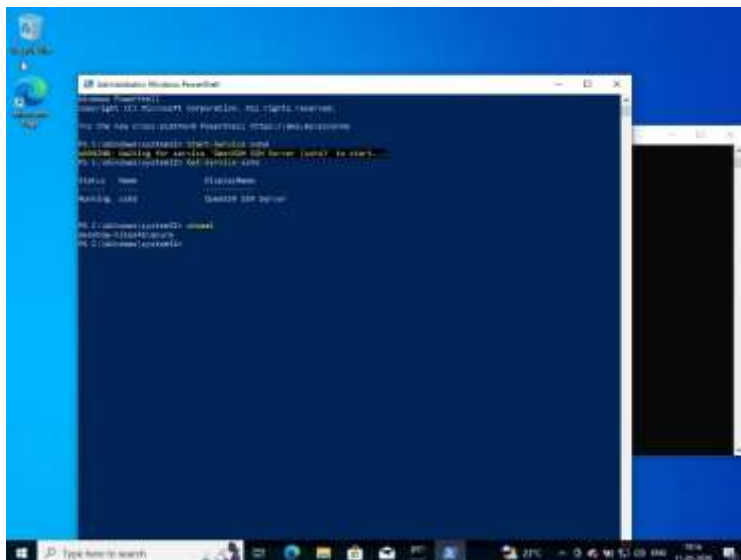
```
powershell id="hkyu6n"
```

```
Get-Service sshd
```

The output shows the service status as **Running**, confirming that the Windows system was ready to accept SSH login attempts.

The whoami command was also used to verify the active username (anura), which was later used in the Hydra attack.

This step was required to make the Windows client reachable from Kali Linux for SSH brute-force simulation.



Simulate Brute-Force Attack Using Hydra

To test the detection system, a brute-force login attack was simulated from the Kali Linux attacker machine using **Hydra**. Hydra is a password-cracking tool commonly used in penetration testing to perform login brute-force attacks against services such as SSH, FTP, HTTP, and others.

In this setup, Hydra was used to target the Windows 10 client machine over the SSH service. The attacker machine attempted multiple password combinations against a valid username using a large wordlist. This generated repeated failed login attempts, which were captured in the Windows Security logs and forwarded to Splunk for monitoring.

The following Hydra command was used:

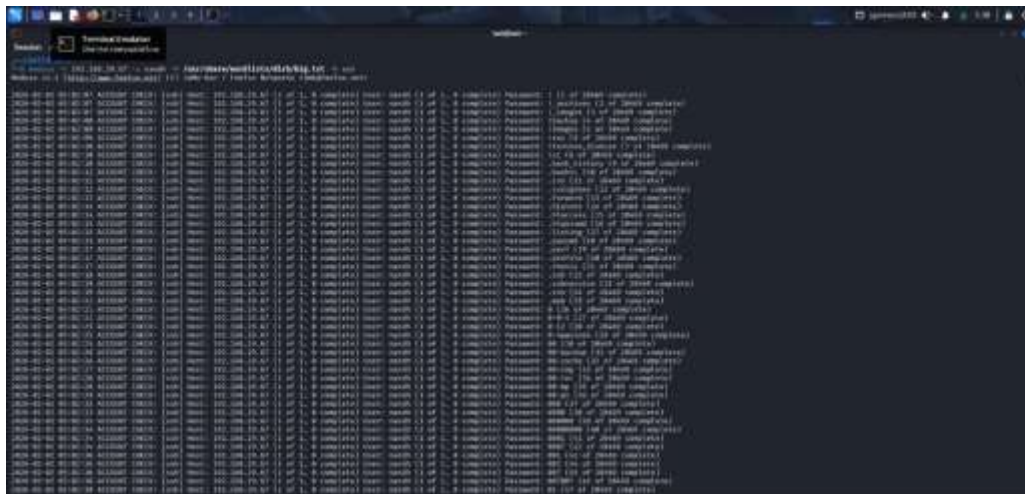
```
bash
hydra -l nandh -P /usr/share/wordlists/dirb/big.txt -t 4 ssh://192.168.29.67
```

Command Explanation

- * hydra → Launches the Hydra brute-force tool
- * -l nandh → Specifies the target username
- * -P /usr/share/wordlists/dirb/big.txt → Uses a password wordlist for brute-force attempts
- * -t 4 → Runs 4 parallel login attempts
- ssh://192.168.29.67 → Targets the SSH service on the Windows client machine

Hydra continuously tested multiple passwords from the wordlist against the target account. Each failed login generated an authentication failure event on the Windows system. These repeated failures were recorded in the Security Event Log and forwarded to Splunk through the Universal Forwarder.

This attack simulation successfully produced the brute-force activity needed to test Splunk's detection, alerting, and automated response mechanisms.



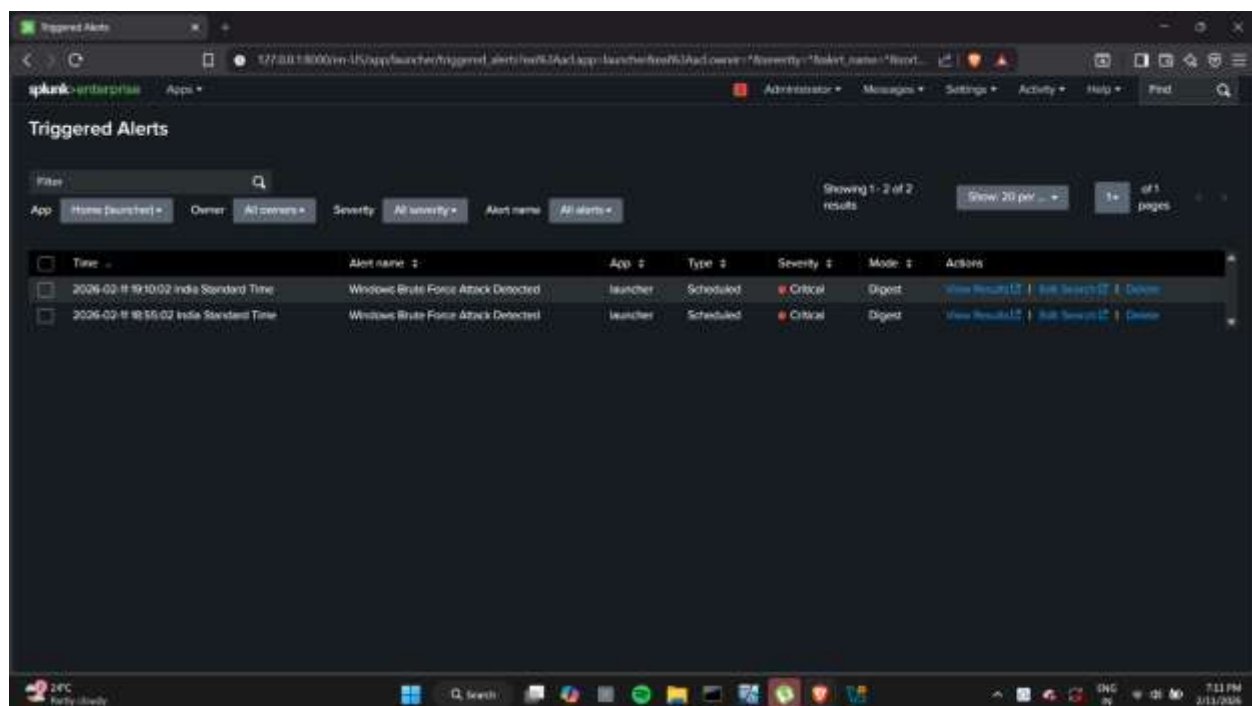
Triggered Alert Detection in Splunk

After the brute-force attack was simulated from the Kali Linux attacker machine, Splunk successfully detected the suspicious login activity and triggered the configured alert. The triggered alerts were visible in the **Triggered Alerts** dashboard of Splunk Enterprise.

The alert named **“Windows Brute Force Attack Detected”** appeared in the alert list, confirming that Splunk identified repeated failed login attempts on the Windows client system. Each alert entry showed the time of detection, alert name, app context, alert type, severity, and available actions.

The alert was triggered as a **Scheduled Alert**, and its severity was marked as Critical, indicating that the failed login activity matched the brute-force detection rule. The **Digest** mode confirmed that the alert was generated based on the scheduled search query and its matching results.

This result confirmed that Splunk successfully monitored the forwarded Windows Security logs, detected multiple



Conclusion

This project demonstrated how Splunk can be used in a virtual lab environment to detect brute-force login attempts on a Windows system. By collecting logs, monitoring failed login events, and creating alerts, the system was able to identify suspicious login activity effectively. The setup also showed how automated response actions, such as blocking attacker IP addresses, can improve system security.

Future Enhancements

- * Add email notifications for instant alerting
- * Detect other attacks such as DoS and port scanning
- * Improve automated response with advanced blocking rules
- * Build a dashboard for real-time attack monitoring